

# Efficiently Counting Triangles for Hypergraph Streams by Reservoir-Based Sampling

Lingling Zhang , Zhiwei Zhang , Guoren Wang , Ye Yuan , and Kangfei Zhao

**Abstract**—Hypergraph streams provide an efficient model to express and preserve complex connections in various applications. Triangles in a hypergraph can be formed by vertices and hyperedges. The specific counts of triangles are important to analyze various applications. Due to the huge costs of counting triangles based on the whole datasets, a sampling-and-estimating framework has low overhead while obtaining a relatively accurate result. However, existing sampling algorithms focus on pairwise graph streams, they estimate the counts of triangles formed by vertices with large estimation errors and can not be applied to count triangles formed by hyperedges. Therefore, this paper first proposes a sampling-and-estimating framework that produces hyperedge samples using a reservoir with static capacity to estimate the total counts of triangles by inferring the probabilities of forming the triangles respectively. Furthermore, to improve the estimation accuracy, this paper proposes another sampling-and-estimating framework to produce samples in the form of hyperedge pairs which can be used to compute the probabilities of the formations of triangles more accurately and then estimate the total triangle counts with smaller estimation variances. The extensive experiments based on real-world datasets confirm the efficiency and accuracy of our proposed frameworks for counting triangles in different types of hypergraphs at a small sampling ratio.

**Index Terms**—Hypergraph streams, triangles counting, reservoir-based sampling, social networks.

## I. INTRODUCTION

In our era of the internet of everything, relations in various applications that involve a group of entities are ubiquitous [1], [2]. As a result, a pairwise graph becomes inefficient to model the relation-based applications as an edge in the pairwise graph only describes the relationship between two vertices (entities).

Manuscript received 13 February 2022; revised 26 November 2022; accepted 5 January 2023. Date of publication 12 January 2023; date of current version 6 October 2023. The work of Zhiwei Zhang was supported in part by the National Key Research and Development Program of China under Grants 2022YFB2702100 2020YFB1707900, 2021YFB2700700, in part by the National Natural Science Foundation of China under Grant 62072035, in part by the Open Research Projects of Zhejiang Lab under Grant 2020KE0AB04, and in part by CCF-Huawei Database System Innovation Research Plan under Grant CCF-HuaweiDBIR2021007B. The work of Ye Yuan was supported in part by the NSFC under Grants 61932004, 62225203, and U21A20516. The work of Guoren Wang was supported by the NSFC under Grants 61732003 and U2001211. Recommended for acceptance by R. C.-W. Wong. (Corresponding authors: Guoren Wang; Zhiwei Zhang.)

Lingling Zhang, Zhiwei Zhang, Guoren Wang, and Ye Yuan are with the Beijing Institute of Technology, Haidian 100811, China (e-mail: llzh@bit.edu.cn; zwzhang@bit.edu.cn; wanggr@bit.edu.cn; yuan-ye@bit.edu.cn).

Kangfei Zhao is with the Tencent AI Lab, Machine Learning Center, Bellevue, WA 98004 USA (e-mail: kfzhao@se.cuhk.edu.hk).

Digital Object Identifier 10.1109/TKDE.2023.3236335

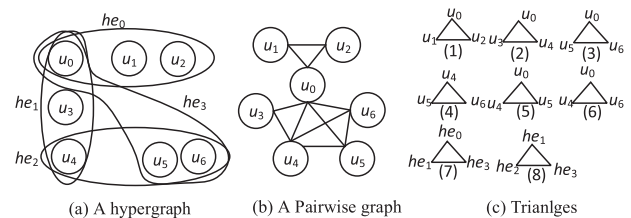


Fig. 1. An example for the triangles in a hypergraph as the vertices in a circle represent a hyperedge.

Fortunately, hypergraphs in which a hyperedge can include an arbitrary number of interacted vertices support to describe interactions among entities in the application in a more generalized and efficient way [3], [4]. Hypergraph streams consist of hyperedges that conveniently express and further preserve interaction information among vertices. Therefore, it is necessary to analyze data based on hypergraph streams for supporting various applications.

Different from the formation of triangles in pairwise graphs in which a triangle is constituted of three completely connected vertices [5], [6], [7], a triangle in a hypergraph is formed either by three vertices or three hyperedges. To count triangles formed by vertices, it is necessary to learn connections among vertices in hypergraphs. Many researchers consider that the vertices in a hyperedge are connected with each other [8]. Thus, a hypergraph can be converted into a corresponding pairwise graph that describes connections among vertices intuitively. For example, the pairwise graph shown in Fig. 1(b) is generated by the hypergraph shown in Fig. 1(a). Then, the triangles formed by vertices are clearly figured out by the front six triangles shown in Fig. 1(c). Furthermore, we notice that each of the front four triangles are formed by vertices which are included by the same hyperedges while the latter two of the six triangles are formed by the vertices which stem from different hyperedges. The last two triangles in Fig. 1(c) are formed by hyperedges.

Accordingly, triangles in a hypergraph can be categorized into three types described as follows.

- 1) *Inner triangles*: If three vertices are included by a hyperedge in a hypergraph, they must form an inner triangle as shown (1), (2), (3) and (4) in Fig. 1(c). Then,  $n$  ( $n \geq 3$ ) vertices in a hyperedge form  $C_n^3$  inner triangles.
- 2) *Outer triangles*: As the counterpart of the inner triangles, if three vertices can not be included by a hyperedge while they are connected with each other in the corresponding

pairwise graph of the hypergraph, they form an outer triangle as shown (5) and (6) in Fig. 1(c).

- 3) *He-Triangles*: If any two of three hyperedges are connected by sharing some common vertex, the three hyperedge thus form a hyperedge triangle (also called He-Triangle) as shown (7) and (8) in Fig. 1(c).

Among the data-analytic tasks, counting triangles plays an importantly fundamental role in the following applications described as below.

*Computing Cluster Coefficient*: Three types of triangles reflect the connectivity for a hypergraph in different aspects [9]. Thus, the count of inner triangles is used to compute the global cluster coefficient for the hypergraph from the angle of vertices. The higher local cluster coefficient means that the larger mean size of a hyperedge. The count of outer triangles can be employed to compute the cluster coefficient showed by the common vertices shared among hyperedges and the higher cluster coefficient computed by the counts of outer triangles means the larger number of vertices participating in more than one hyperedge. The count of He-Triangles is used to compute the global cluster coefficient from the angle of the connection among hyperedges.

*Detecting Community*: Triangles are the basic unit for detecting the community for a hypergraph. In a hyperedge stream, if a hyperedge participates many triangle formations, it has a higher chance for participating a cohesive community. Furthermore, finding K-truss as a subgraph model is important for detecting the community while it requires the counts of triangles [10], [11].

*Providing Important and Powerful Information in Many Fields*: Besides for the above applications which use the counts of triangles directly, many applications can leverage the counts as auxiliary tools or important parameters. (1) *Influence estimation* in social networks [12], [13]. For example, when to estimate the spread of a virus in a human network modeled by a hypergraph, the persons in the inner triangles represent they are family members while those in the outer triangles show they are relatives and those in He-Triangles may reflect they share some common properties, i.e., taking part in the same interest activity. Thus, these vertices in different types of triangles have different infectivities to the others. (2) *Congestion control* in computer networks [14], [15]. Inner triangles in computer networks represent the computers share a common switch while outer triangles are formed by the computers communicated by different switches, and He-Triangles are to represent the communications among different switches. The counts of different types of triangles mean to what extent the congestion will affect and thus a proper plan of congestion control will be strategically executed. Therefore, when counting triangles in a hypergraph, it is necessary to compute the specific number of the three types of triangles respectively.

Due to the huge volume of data, it is highly resource-consuming to count the triangles over the whole data [16], [17], [18]. Since an approximate answer to the total count of triangles is usually enough in many fields especially when the estimation results can be proofed unbiased and convincingly, sampling-and-estimating frameworks are employed to provide an efficient and effective tools [19], [20], [21]. However, extensively previous

works focus on counting triangles over pairwise graph streams formed by edges [21], [22], [23] and there is no existing work on counting the triangles over hypergraph streams. A direct approach to counting triangles over hypergraph streams is to first transform a hypergraph into a pairwise graph and then use the sophisticated sampling and estimating algorithms to count triangles. However, such type of approaches has the following three problems in dealing with large hypergraph streams.

*Indistinguishable of the Three Types of Triangles*: Existing sampling-and-estimating frameworks do not care about whether the vertices forming triangles are contained by the same hyperedges or not [22], [23], [24]. Furthermore, the existing sampling-and-estimating cannot figure out the triangles formed by the connected hyperedges because they produce edge samples [25], [26], [27]. Therefore, these approaches can not be applied in the applications which require to differentiate the triangles as described above.

*Large Estimation Costs*: When a hypergraph stream is transformed into a pairwise graph, a hyperedge containing  $k$  vertices generates  $C_k^2$  edges in the pairwise graph [2], [28]. Thus, the number of edges is much larger than that of hyperedges. Thus, given a small memory size, the sampling techniques may not produce enough samples to form an adequate number of triangle samples for estimating the total triangle counts. Thus, a larger number of samples is required, resulting in the increase of both the processing time and memory cost.

*Large Estimation Errors*: If a hyperedge include more than two vertices, it corresponds to several edges in a pairwise graph. Then, if two vertices are contained by two hyperedges, the identical edges are produced repeatedly in the pairwise graph stream. However, the existing sampling-and-estimating frameworks deal with these duplicate edges without consideration on the source formations of triangles. They set the probability of each unique edge as equal, resulting in inaccurately inferring the probability of forming triangles by duplicate edges. Consequently, their estimated counts suffer from large estimation errors.

To overcome these problems, this paper proposes new sampling and estimating frameworks named *HyperSV* and *HyperSHE* respectively to be adapt in various applications. *HyperSV* produces hyperedge samples from which the formation probabilities of the three types of triangles are inferred respectively to count the total triangles. Since *HyperSV* samples a hypergraph stream directly without any transformations, it is able to provide real-time estimations. On the other hand, to improve the accuracy of *HyperSV*, *HyperSHE* optimizes the sampling process by producing samples in the form of hyperedge pairs that can be used to infer the probabilities of the three types of triangles more accurately and thus estimate the total counts of triangles with smaller estimation variances than *HyperSV*. With the design and implementation of counting the triangles for hypergraphs by *HyperSV* and *HyperSHE*, this paper makes the following four contributions.

- *Frameworks*: To the best of our knowledge, we are the first to propose sampling-and-estimating frameworks for counting the three types of triangles over the hypergraphs simultaneously. *HyperSV* can estimate the triangles at

TABLE I  
THE DEFINITIONS USED IN THIS PAPER

|                                |                                                                                        |
|--------------------------------|----------------------------------------------------------------------------------------|
| $Hg = \{V, He\}$               | hypergraph stream $Hg$                                                                 |
| $V$                            | vertex set in $Hg$                                                                     |
| $He$                           | hyperedge set in $Hg$                                                                  |
| $G_v = \{V, E_v\}$             | vertex-based graph of $Hg$                                                             |
| $G_h = \{V_h, E_h\}$           | hyperedge-based graph of $Hg$                                                          |
| $V(he)$                        | the set of vertices included by hyperedge $he$                                         |
| $deg(he)$                      | the number of neighboring hyperedges of $he$                                           |
| $ S $                          | the number of samples in the sample set $S$                                            |
| $p_i$                          | sampling probability when the $i^{th}$ hyperedge is dealt with in $Hg$                 |
| $p_i(ave)$                     | mean sampling probability of an edge when the $i^{th}$ hyperedge is dealt with in $Hg$ |
| $p_i(e)$                       | sampling probability when the $i^{th}$ hyperedge pair is dealt with in $G_h$           |
| $I_{\Delta}/I_{\Delta}^i(e)$   | total count of inner triangles or the $i^{th}$ hyperedge pair in $G_h$                 |
| $O_{\Delta}/O_{\Delta}^i(e)$   | total count of outer triangles or the $i^{th}$ hyperedge pair in $G_h$                 |
| $HE_{\Delta}/HE_{\Delta}^i(e)$ | total count of He-Triangles or the $i^{th}$ hyperedge pair in $G_h$                    |
| $Cov(\Delta_0, \Delta_1)$      | the covariance between $\Delta_0$ and $\Delta_1$                                       |

real time while *HyperSHE* estimates the three types of triangles with smaller estimation errors.

- *Sampling processes*: We employ reservoir-based sampling in *HyperSV* to obtain hyperedge samples while obtaining samples in the form of hyperedge pairs by converting a hypergraph stream into a hyperedge-based pairwise graph that is able to reflect the connectivity among hyperedges.
- *Estimating triangles*: We propose effective estimation algorithms to accommodate with the sampling processes to approximately count inner triangles, outer triangles and He-Triangles accurately by inferring the probabilities of forming three completely connected edges or hyperedges.
- *Experimental demonstrations*: We conduct extensive experiments to show the efficiency of our proposed frameworks in terms of estimation errors and variances, processing time and occupied memory when *HyperSV* and *HyperSHE* are used to estimate the three types of triangles.

The rest of the paper is organized as follows. Section II introduces background and motivation that stimulates our proposed frameworks. Section III describes a new sampling and estimating framework by producing hyperedge samples to obtain the three types of triangles while Section IV optimizes the sampling and estimating framework by producing hyperedge pairs. Section V discusses the applications of the two frameworks while Section VI describes the experimental results to confirm the efficiency of our proposed frameworks. Section VII reviews the related works of our work while Section VIII summarizes our work.

## II. PRELIMINARY

This section describes preliminary which is important to design the reservoir-based sampling methods for counting triangles in hypergraphs. More specifically, we first introduce the definitions related to a hypergraph and a pairwise graph which will be used in the whole paper. Then, we describe the problem that this paper deals with and the baseline sampling methods

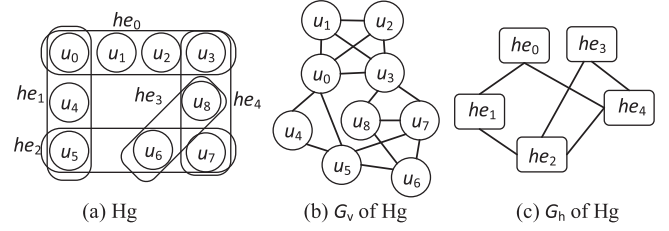


Fig. 2. An example for a hypergraph ( $Hg$ ) and its vertex-based pairwise graph ( $G_v$ ) and hyperedge-based pairwise graph ( $G_h$ ).

to count the triangles for the hypergraphs. Table I describes the frequently-used symbols in this paper.

### A. Definition

*Hypergraph and Pairwise Graph*: We use  $Hg = \{V, He\}$  denote a hypergraph, where  $V$  is the set of vertices and  $He$  is the set of hyperedges in  $Hg$ . As many researchers suggest that the vertices in the same hyperedge can be considered as connected, the hypergraph can be transformed into a pairwise graph to show the specific connections between any two vertices. We consider the transformed pairwise graph as  $G_v = \{V, E_v\}$ , also named vertex-base pairwise graph of  $Hg$ , where  $E_v$  denotes the edge set of  $G_v$ . For example, a hypergraph  $Hg$  shown in Fig. 2(a) can be transformed into a vertex-based pairwise graph  $G_v$  shown in Fig. 2(b). We further use a hyperedge-based pairwise graph  $G_h = \{V_h, E_h\}$  to describe the connections among hyperedges, where  $V_h$  denotes the vertex set formed by hyperedges and  $E_h$  denote edge set formed by the hyperedge pairs each of which share some common vertex. Fig. 2(c) shows the specific connection between two hyperedges of Fig. 2(a). We use  $V(he)$  denote the vertices included by  $he$ . If  $V(he_1) \cap V(he_2) \neq \emptyset$ , we consider the two hyperedges  $he_1$  and  $he_2$  are connected. If  $he_0$  connects to  $he_1$ ,  $he_1$  is considered as one neighbor of  $he_0$ . We use  $deg(he)$  denote the number of  $he$ 's neighboring hyperedges.

*Inner and Outer Triangles*: If three vertices  $\mu_0, \mu_1$ , and  $\mu_2$  are included by the same hyperedge, then they form an inner triangle. If three vertices (i.e.,  $\mu_0, \mu_1$  and  $\mu_2$ ) are shared by three hyperedge (i.e.,  $he_0, he_1$  and  $he_2$ ) as that:  $\mu_0 \in V(he_0) \cap V(he_1)$ ,  $\mu_1 \in V(he_1) \cap V(he_2)$  and  $\mu_2 \in V(he_0) \cap V(he_2)$ , then we consider the three vertices form an outer triangle.

*He-Triangles*: If there are three hyperedges where  $he_1, he_2, he_3 \in He$ ,  $(he_1, he_2), (he_2, he_3), (he_1, he_3) \in E_h$  in the hyperedge-based pairwise graph, the three hyperedges form a hyperedge triangle.

### B. Baseline Method

*Problem Statement*: A hypergraph stream  $Hg$  consists of hyperedges labeled as  $\{he_1, \dots, he_n\}$ , where  $n$  is the total number of hyperedges in the stream. A sampling method is employed to produce samples that is a subgraph  $Hg^c$  of  $Hg$  and  $Hg^c$  is used to estimate the triangles of  $Hg$ . When using a sampling algorithm to estimate the total counts of the three types of triangles, the key factor is to obtain the probability of a triangle formed by the samples. Let  $p_{\Delta}$  denote the probability of forming a triangle

**Algorithm 1: Baseline Method.**


---

**Input:** A hypergraph stream  $Hg = \{he_1, \dots, he_f\}$ , a static reservoir  $R_s$ .

**Output:** The estimated counts of triangles.

```

1  $G_v \leftarrow Transform(Hg)$ ;
2  $G_s \leftarrow sampleEdge(G_v)$ ;
3  $\Delta_s \leftarrow computeTriangle(G_s)$ ;
4 for  $\Delta_i \in \Delta_s$  do
5   if  $\Delta_i$  is an inner triangle then
6      $\Delta_{inner} \leftarrow \Delta_{inner} + 1$ ;
7   else
8      $\Delta_{outer} \leftarrow \Delta_{outer} + 1$ ;
9  $I_{triangle}^i \leftarrow \frac{\Delta_{inner}}{p_{\Delta}^i}$ ;
10  $O_{triangle}^i \leftarrow \frac{\Delta_{outer}}{p_{\Delta}^o}$ ;

```

---

from samples, the total count of triangles ( $Total_{\Delta}$ ) is estimated as  $Total_{\Delta} = \frac{h}{p_{\Delta}}$ , where  $h$  is the number of triangles formed by samples.

**Baseline Method:** Since the existing sampling methods on pairwise graphs are widely studied [21], [25], [29], [30] and a hypergraph can be easily converted into the corresponding pairwise graph, a baseline method is naturally proposed based on the existing methods. Thus, the baseline sampling method includes four steps: (1) Transforming hypergraphs into pairwise graphs; (2) Obtaining edge samples by the existing sampling methods; (3) Discovering the triangles formed by these samples and justifying the type of these triangles; (4) Estimating the total triangles by using the sampled triangles and its obtained sampling probability. Such a baseline sampling method is described in Algorithm 1, that can leverage the existing state-of-the-art sampling methods on pairwise graphs to estimate the triangle counts in the hypergraphs.

Although Algorithm 1 is able to estimate the count of triangles formed by vertices simple by employing the sampling probability defined in the existing sampling methods, it fails to obtain the sampling probability of hyperedges each of which include an arbitrary number of vertices and it cannot estimate the count of triangles formed by hyperedges. When transforming hypergraphs into pairwise graphs, the total number of edges is much larger than that of hyperedges, and thus more samples are needed to estimate the triangle counts, consuming more resources. Furthermore, it is very time-consuming to search the whole hyperedges for determining whether a sampled triangle belongs to an inner one or an outer one from their definitions. Therefore, it is necessary to develop sampling and estimating methods based on the hypergraphs to obtain the counts of various types of triangles.

### III. HYPEREDGE-BASED SAMPLING AND ESTIMATING

This section describes a hyperedge-based sampling and estimation framework that consist of two parts: a sampling algorithm and its estimation counterpart. We first describe the sampling algorithm by elaborating the sampling probability, infer the

**Algorithm 2: HyperSV.**


---

**Input:** A hypergraph stream  $Hg = \{he_1, \dots, he_f\}$ , a static reservoir  $R_s$ .

**Output:** A set of hyperedge samples  $R_s$ .

```

1  $R_s \leftarrow \{he_1, \dots, he_c\}$ ;
2  $i \leftarrow c + 1$ ;
3 for  $he_i \in Hg$  do
4    $p_i^s = \frac{c}{i}$ ;
5   Generate  $p_r$  randomly from  $(0, 1]$ ;
6   if  $p_r \leq p_i^s$  then
7     Generate  $rand\_pos$  randomly from  $[0, c]$ ;
8      $R_s \leftarrow R_s - R_s[rand\_pos]$ ;
9      $R_s \leftarrow R_s \cup he_i$ ;

```

---

probabilities of forming the three types of triangles respectively which are used to approximately compute the corresponding triangle counts and analyze the estimation variances.

#### A. Hyperedge-Based Sampling Algorithm

1) *Reservoir-based hyperedge sampling: HyperSV:* Reservoir-based hyperedge sampling called *HyperSV*, deals with a hyperedge stream directly without any transformations. It is a general idea of reservoir-based sampling for dealing with hypergraphs directly. HyperSV employs a reservoir with a static capacity to preserve hyperedge samples. Given a reservoir with a static repository, i.e.,  $c$ , the probability for the  $i^{th}$  ( $i > c$ ) hyperedge entering into the reservoir is  $p_i^s = \frac{c}{i}$  as the front  $c$  hyperedges are preserved in the reservoir directly. Before the  $i$ th hyperedge is preserved in the reservoir, a hyperedge that has been stored in the reservoir should be removed with probability  $p_o = \frac{1}{i}$  to ensure the static repository of the reservoir. Therefore, the  $k^{th}$ , ( $k \leq i$ ) hyperedge is stored in the reservoir, meaning that the  $k$ th hyperedge was sampled and it has not been removed in the subsequent sampling process. Thus, the probability  $p_k$  for the  $k$ th hyperedge that is stored in the static reservoir when the *HyperSV* is dealing with the  $i$ th hyperedge, is computed as follows.

$$p_k = \frac{c}{k} \times \left( \frac{c}{k+1} \times \frac{c-1}{c} + \frac{k+1-c}{k+1} \right) \times \dots \times \left( \frac{c}{i} \times \frac{c-1}{c} + \frac{i-c}{i} \right) = \frac{c}{i} \quad (1)$$

Eq. (1) shows that the sampling probability of each hyperedge is irrelevant to its arrival order. We will use the sampling probability of each hyperedge to generate the probability of forming triangles when estimating the three types of triangles detailed later. *HyperSV* is presented in Algorithm 2. When a hyperedge arrives, Lines 4-7 describe the *HyperSV*'s sampling process.

2) *Reservoir-Based Weight Sampling:* As described in (1), the sampling probability of each hyperedge in the stream is equal in the reservoir-based hyperedge sampling. This sampling method does not consider the specific sizes of hyperedges where larger sizes of hyperedge samples will increase the processing time. Therefore, to reduce the sampled hyperedges

with larger sizes, we develop reservoir-based weight sampling, called *HyperWSV*, by considering the sizes of hyperedges as that: the hyperedges with larger sizes have smaller sampling probabilities.

**HyperWSV:** Given a reservoir with fixed capacity  $c$ , we design an algorithm to store more hyperedges with smaller sizes. The algorithm works as that: The algorithm works as that: the first  $c$  hyperedges are stored in the reservoir first and then there are chances to update the hyperedges in the reservoir according to the probabilities set according to the hyperedge sizes. Given the  $i$ th hyperedge, a variable parameter  $\alpha \times s_i$  is employed to tune the sampling probability of the  $i$ th hyperedge. Specifically, when the  $i$ th ( $i > c$ ) hyperedge is being dealt with, the probability for the hyperedge entering into the reservoir is defined as follows, where  $s_i$  is the size of the  $i$ th hyperedge in the hyperedge stream.

$$p_i = \frac{1}{\alpha \times k \times s_i} \quad (2)$$

From (2), if the  $i$ th hyperedge includes more vertices, it has a smaller probability entering into the reservoir. Once the  $i$ th hyperedge entered the reservoir, one of the samples being stored in the reservoir should be removed with a probability  $\frac{1}{c}$ . Thus, although the first  $c$  hyperedges are preserved in the reservoir certainly, they have a chance to be removed from the reservoir in the subsequent sampling. Each of the  $c$  hyperedges is being removed with a probability  $\frac{1}{k \times s_i} \times \frac{1}{c}$  when to deal with the  $i^{th}$  ( $i > c$ ) hyperedge. Therefore, the sampling probability for the  $c$  hyperedge is described as follows.

$$p_i = \left(1 - \frac{1}{\alpha \times k \times s_{c+1}} \times \frac{1}{c}\right) \times \dots \times \left(1 - \frac{1}{\alpha \times k \times s_n} \times \frac{1}{c}\right) \quad (3)$$

Accordingly, the sampling probability for the  $i$ th ( $i \leq c$ ) hyperedge is described as follows.

$$p_i = \frac{1}{\alpha \times k \times s_i} \times \left(1 - \frac{1}{\alpha \times k \times s_{i+1}} \times \frac{1}{c}\right) \times \dots \times \left(1 - \frac{1}{\alpha \times k \times s_n} \times \frac{1}{c}\right) \quad (4)$$

## B. Estimating Triangle Counts

A key factor of using a sampling method to estimating the total triangle counts is to compute the probability of forming a triangle by these samples. We use the sampled hyperedges to estimate the three types of triangle counts in a hypergraph stream respectively.

**Inner triangle estimator:** The vertices in a hyperedge are connected with each other so that  $m$  ( $m \geq 3$ ) vertices in a hyperedge (i.e.,  $he$ ) form  $N(\Delta)_{he} = \frac{m \times (m-1) \times (m-2)}{6}$  triangles. Furthermore, the probability ( $p\Delta_{he}$ ) of these triangles formed by the vertices in a hyperedge is equal to that of the hyperedge and thus  $p\Delta_{he} = \frac{c}{n}$  where  $n$  is the total number of hyperedges in the stream. For a set of hyperedge samples  $S$ , the total counts

of inner triangles are estimated as follows.

$$I_\Delta = \sum_{he \in S} \frac{N(\Delta)_{he}}{p\Delta_{he}} \quad (5)$$

**Theorem 1:** Let  $I(\Delta)$  denote the number of inner triangles for  $n$  hyperedges in  $HG$ , and thus  $I(\Delta) = I_\Delta$ .

**Proof:** Let  $\lambda_{he_i}$  indicate whether the  $i$ th hyperedge  $he_i$  is sampled or not and  $E(I(\Delta)_{he_i})$  denotes the expected number of triangles formed by the vertices in hyperedge  $he_i$ . Thus, the number  $I_{\Delta(he_i)}$  of inner triangles formed by the vertices in hyperedge  $he_i$  is expressed as follows.

$$I_{\Delta(he_i)} = \frac{1}{p(\lambda_{he_i})} \times E(I(\Delta)_{he_i}) \times N(\Delta)_{he_i} \quad (6)$$

Since the probability of producing  $N(\Delta)_{he_i}$  triangles is equal to  $p(\lambda_{he_i})$ , then  $E(I(\Delta)_{he_i}) = p(\lambda_{he_i}) \times N(\Delta)_{he_i}$ . Thus, we have  $I_{\Delta(he_i)} = N(\Delta)_{he_i}$  and then  $I_\Delta = \sum_{he \in S} N(\Delta)_{he} = I_\Delta$ .

Based on (6), the estimation variance  $Var(I_\Delta)$  is given as follows.

$$\begin{aligned} Var(I_\Delta) &= \frac{N(\Delta)_{he}}{p\Delta_{he}^2} \sum_{he_1 \in S} \sum_{he_2 \in S} Cov(he_1, he_2) \\ &= \frac{N(\Delta)_{he}}{p\Delta_{he}^2} \left[ \sum_{he_1 \in S} Var(he_1) \right. \\ &\quad \left. + \sum_{he_1 \in S} \sum_{he_2 \in S, he_1 \neq he_2} Cov(he_1, he_2) \right] \\ &= \frac{N(\Delta)_{he}}{p\Delta_{he}^2} [(p\Delta_{he} - p\Delta_{he}^2) \\ &\quad + \sum_{he_1 \in S} \sum_{he_2 \in S, he_1 \neq he_2} Cov(he_1, he_2)] \quad (7) \end{aligned}$$

If two hyperedges i.e.,  $he_1$  and  $he_2$  share no more than two vertices as  $N_{cv} = |V(he_1) \cap V(he_2)| \leq 2$ , then we have  $Cov(he_1, he_2) = 0$ . Otherwise, the number of the common triangles formed by the vertices in the two hyperedges is  $N(\Delta)_{cv} = C_{N_{cv}}^3$  and  $Cov(he_1, he_2) < N(\Delta)_{cv} \times (p\Delta_{he_1} - p\Delta_{he_1} \times p\Delta_{he_2}) = N(\Delta)_{cv} \times (p\Delta_{he} - p\Delta_{he}^2)$ . Therefore, the accuracy of estimating the inner triangles is influenced by the structures of the connected hyperedges in a hypergraph stream.

**Outer triangle estimator:** As described in Section II, an outer triangle formed by three vertices that stem from different hyperedges. Therefore, to discover outer triangles, the connection among hyperedges should be learned in advance. Theorem 2 explains how to identify an outer triangle in three connected hyperedges.

**Theorem 2:** Given a hypergraph  $H_g$ , and three vertices  $\mu_0$ ,  $\mu_1$  and  $\mu_2$ , they construct an outer triangle if and only if there exist three hyperedges  $he_0, he_1, he_2$ , such that  $\mu_0 \in (V(he_0) \cap V(he_1) - C_v(he_0, he_1, he_2))$ ,  $\mu_1 \in (V(he_1) \cap V(he_2) - C_v(he_0, he_1, he_2))$  and  $\mu_2 \in (V(he_0) \cap$

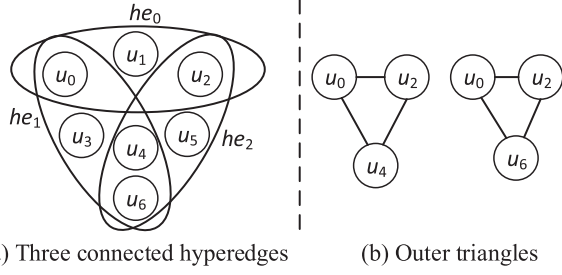


Fig. 3. The connected hyperedges and their corresponding outer triangles.

$V(he_2) - C_v(he_0, he_1, he_2)$ ), where  $C_v(he_0, he_1, he_2)$  as the set of vertices shared by the three hyperedges.

*Proof.* If  $(\mu_0, \mu_1, \mu_2)$  forms an outer triangle, it means there are three connected edges as  $\{(\mu_0, \mu_1), (\mu_0, \mu_2), (\mu_1, \mu_2)\}$  when a hypergraph stream is converted to a pairwise graph. If two edges are produced by the vertices included by a hyperedge, i.e.,  $(\mu_0, \mu_1), (\mu_0, \mu_2)$  derived from  $he$  and then  $\mu_0, \mu_1, \mu_2 \in V(he)$ , then the third edge can be certainly produced by the hyperedge. Therefore, the three edges forming an outer triangle must derive from three different hyperedges. For example, in Fig. 3(a), three edges formed by the vertices in  $\{\mu_0, \mu_2, \mu_4\}$  are scattered in the three hyperedges:  $(\mu_0, \mu_2)$  in  $he_0$ ,  $(\mu_0, \mu_4)$  in  $he_1$  and  $(\mu_2, \mu_4)$  in  $he_2$  respectively. The case for the triangle formed by  $\{\mu_0, \mu_2, \mu_6\}$  is the same as for that formed by  $\{\mu_0, \mu_2, \mu_4\}$  as shown in Fig. 3(b). Correspondingly, assume  $\mu_0 \in (V(he_0) \cap V(he_1) - C_v(he_0, he_1, he_2))$ ,  $\mu_1 \in (V(he_0) \cap V(he_2) - C_v(he_0, he_1, he_2))$  and  $\mu_2 \in (V(he_1) \cap V(he_2) - C_v(he_0, he_1, he_2))$ , thus we have  $\mu_0 \in V(he_0)$ ,  $\mu_1 \in V(he_0)$  and  $(\mu_0, \mu_1)$  is an edge. Likewise,  $(\mu_0, \mu_2)$  and  $(\mu_1, \mu_2)$  are the two edges formed by another two hyperedges and thus the three edges form an outer triangle.

Accordingly, Theorem 2 provides an approach to counting outer triangles in a hypergraph stream. If three hyperedges e.g.,  $he_0, he_1$  and  $he_2$  are connected and the number of the shared vertices by any hyperedge pairs are denoted as  $N_{he_0, he_1}$ ,  $N_{he_0, he_2}$  and  $N_{he_1, he_2}$  respectively and the number of the vertices shared by the three hyperedges is  $com_v$ , the counts of the triangles are denoted as  $O(\Delta_{(he_0, he_1, he_2)}) = (N_{he_0, he_2} - com_v) \times (N_{he_0, he_2} - com_v) \times (N_{he_1, he_2} - com_v)$ .

Thus, it is necessary to infer the probability of generating a pair of hyperedges that are connected by some common vertices. Let  $deg(he)$  denote the number of hyperedges that have connections to hyperedge  $he$ , and thus there are  $deg(he)$  times in the hyperedge stream to obtain a neighboring hyperedge for  $he$ . Since  $he$  is sampled with the probability  $p_i$  when *HyperSV* is dealing with the  $i$ th hyperedge, the probability  $p_i(he\_pair)$  to obtain one of its connected hyperedges is  $p_i(he\_pair) = p_i \times deg(he)$ . However, it is impossible to learn the number of neighboring hyperedges for a hyperedge without transforming the hyperedge stream into a hyperedge-based pairwise graph. In actual operations, we use the average number  $aveDeg$  of the neighboring hyperedges for a hyperedge in the hyperedge samples to replace for  $deg(he)$  as:  $aveDeg = \frac{totalP}{|S|}$  where  $totalP$  refers to the total number of hyperedge pairs each of which

are connected and  $|S|$  is the number of sampled hyperedges. Thus, the probability of forming a hyperedge pair is computed as  $p_i(ave) = p_i \times aveDeg$  and the expected estimation counts of triangles using *HyperSV* are given as follows.

$$E(O_{\Delta}) = \frac{\sum_{he_0, he_1, he_2 \in S^t, he_0 \neq he_1 \neq he_2} O(\Delta_{(he_0, he_1, he_2)})}{(p_i(ave))^3} \quad (8)$$

**Theorem 3:** Let  $\lambda_{3he}^o$  denote the true number of outer triangles formed by three completely connected hyperedges, and thus  $\lambda_{3he}^o = E(O(\Delta_{3he}^o))$  where  $O(\Delta_{3he}^o)$  denotes the number of triangles because of the three hyperedges through sampling.

*Proof:* Since the probability of a hyperedge pair is  $p_i(ave)$  and an outer triangle is formed by three completely connected hyperedge pairs, then the sampling probability of the outer triangle because of the three hyperedges is  $(p_i(ave))^3$ . Thus, the expected number of the outer triangles obtained by three hyperedges through sampling is  $E(O(\Delta_{3he}^o)) = O(\Delta_{3he}^o) \times (p_i(ave))^3 = \lambda_{3he}^o$ .

Let  $S(\Delta_o)$  denote the outer triangles formed by the three hyperedge pairs deriving from sample set  $S$ . Then, the estimation variance  $Var(O_{\Delta})$  is expressed as follows.

$$\begin{aligned} Var(O_{\Delta}) &= Var\left(\frac{\sum_{\lambda_{\Delta_0} \in S(\Delta_o)} \lambda_{\Delta_0}}{(p_i^t(ave))^3}\right) \\ &= Var\left(\frac{1}{(p_i^t(ave))^3} \sum_{\lambda_{\Delta_0} \in S(\Delta_o)} \lambda_{\Delta_0}\right) \\ &= \frac{1}{(p_i^t(ave))^6} \sum_{\lambda_{\Delta_0} \in S(\Delta_o)} \sum_{\lambda_{\Delta_1} \in S(\Delta_o), \lambda_{\Delta_0} \neq \lambda_{\Delta_1}} Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) \\ &= \frac{1}{(p_i^t(ave))^6} \left( \sum_{\lambda_{\Delta_0} \in S(\Delta_o)} \right. \\ &\quad \left. + \sum_{\lambda_{\Delta_0} \in S(\Delta_o)} \sum_{\lambda_{\Delta_1} \in S(\Delta_o), \lambda_{\Delta_0} \neq \lambda_{\Delta_1}} Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) \right) \quad (9) \end{aligned}$$

If two outer triangles, i.e.,  $\Delta_0$  and  $\Delta_1$  are generated by the vertices stemming from different hyperedges, then we have  $Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) = 0$ . Otherwise, two following cases lead to  $Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) > 0$ .

- Two outer triangles, i.e.,  $\Delta_0$  and  $\Delta_1$  consist of a common vertex which is shared by two hyperedges. For example, as shown in Fig. 4(a), an outer triangle  $\Delta(u_0, u_5, u_7)$  is formed by vertices in three hyperedges  $\{he_1, he_2, he_3\}$  and another outer triangle  $\Delta(u_0, u_6, u_2)$  is formed by the vertices in another group of three hyperedges  $\{he_0, he_2, he_4\}$ . In this case, we have  $Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) = (p_i(ave))^5 - (p_i(ave))^6$ .
- Two outer triangles, i.e.,  $\Delta_0$  and  $\Delta_1$  consist of two common vertices each of which is shared by two hyperedges. For example, Fig. 4(b) shows two outer triangles:  $\Delta(u_0, u_1, u_4)$  and  $\Delta(u_1, u_2, u_4)$  that share two vertices. In this case, we have  $Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) = (p_i(ave))^4 - (p_i(ave))^6$ .

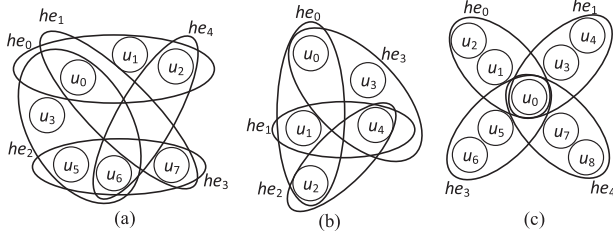


Fig. 4. An example of the connected hyperedges: (a) and (b) form relevant outer triangles while (a), (b) and (c) form relevant He-Triangles.

**He-Triangle estimator:** Recall from Section II, if three hyperedges are connected, they can form an hyperedge triangle. It is noteworthy that the three connected hyperedges must form a hyperedge triangle while they may not form outer triangles. For example,  $he_0$ ,  $he_1$  and  $he_2$  are connected while they just share a common vertex, and then  $he_0$ ,  $he_1$  and  $he_2$  can form a hyperedge triangle but there is no outer triangle derived from the three hyperedges. Therefore, the sample set  $S$  is also used to estimate He-Triangles.

In sampling process of *HyperSV*, while each hyperedge is sampled with probability  $p_i$ , we consider the probability of obtaining a pair related to a sampled hyperedge as  $p_i(ave) = p_i \times aveDeg$  and the probability of forming a He-Triangle by three hyperedges can be infer as  $(p_i(ave))^3$ . Thus, the expected value of He-Triangle counts is expressed as follows, where function  $1(he_0, he_1, he_2)$  describes that whether  $he_0$ ,  $he_1$  and  $he_2$  can form a He-Triangle. If so,  $1(he_0, he_1, he_2) = 1$ ; Otherwise,  $1(he_0, he_1, he_2) = 0$ .

$$E(HE_{\Delta}) = \frac{\sum_{he_0, he_1, he_2 \in S, he_0 \neq he_1 \neq he_2} 1(he_0, he_1, he_2)}{(p_i(ave))^3} \quad (10)$$

Eq. (10) can be proofed to estimate the total counts of He-Triangles and the specific proof is similar to that of (8) explained in Theorem 3. Due to the space limits, we omit the proof. Let  $S(\Delta_{he})$  denote He-Triangles formed by the hyperedges in the sample set  $R_s$ . The computation of the estimation variance for  $E(HE_{\Delta})$  is similar to that for  $E(O_{\Delta})$  as follows.

$$Var(HE_{\Delta}) = \frac{1}{(p_i(ave))^6} \left( \sum_{\lambda_{\Delta_0} \in S(\Delta_{he})} \sum_{\lambda_{\Delta_1} \in S(\Delta_{he}), \lambda_{\Delta_0} \neq \lambda_{\Delta_1}} Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) \right) \quad (11)$$

Similar to  $Var(O_{\Delta})$ , if two He-Triangles i.e.,  $\Delta_0$  and  $\Delta_1$  are formed by six different hyperedges,  $Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) = 0$ . Since each outer triangle must corresponds to a He-Triangle and the counterpart is not valid, the connective cases of the two He-Triangles include those of the two outer triangles. Besides, there is another case that that leads to  $Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) > 0$  as that two HE-Triangles are formed by four hyperedges sharing

---

**Algorithm 3:** *HyperSV\_Compute*( $\Delta$ ).

---

**Input:** Sample set  $S$

**Output:**  $I_{\Delta}$ ,  $O_{\Delta}$  and  $HE_{\Delta}$

```

1 for  $he \in S$  do
2    $m \leftarrow |V(he)|$ ;
3   if  $m \geq 3$  then
4      $I_{\Delta}^i \leftarrow I_{\Delta}^i + \frac{m \times (m-1) \times (m-2)}{3 \times 2}$ ;
5 for  $he_j, he_t, he_r \in S$  do
6   if  $he_j, he_t, he_r$  form a hyperedge triangle then
7      $HE_{\Delta}^i \leftarrow HE_{\Delta}^i$ ;
8      $O_{\Delta}^i \leftarrow O_{\Delta}^i + enum_{\Delta}(he_j, he_t, he_r)$ ;
9 Procedure  $enum_{\Delta}(he_j, he_t, he_r)$ ;
10   $C_v(he_j, he_t, he_r) \leftarrow V(he_j) \cap V(he_t) \cap (V(he_r))$ ;
11   $n_1 \leftarrow V(he_j) \cap V(he_t) - C_v(he_j, he_t, he_r)$ ;
12   $n_2 \leftarrow V(he_t) \cap V(he_r) - C_v(he_j, he_t, he_r)$ ;
13   $n_3 \leftarrow V(he_j) \cap V(he_r) - C_v(he_j, he_t, he_r)$ ;
14   $O_{\Delta}^t \leftarrow O_{\Delta}^t + n_1 \times n_2 \times n_3$ ;
15   $I_{\Delta} = \frac{I_{\Delta}^i}{p_i}$ ;
16   $O_{\Delta} = \frac{O_{\Delta}^i}{(p_i(ave))^3}$ ;
17   $HE_{\Delta} = \frac{HE_{\Delta}^i}{(p_i(ave))^3}$ ;

```

---

at least a common vertex. As shown in Fig. 4(c), two triangles  $\Delta(he_0, he_1, he_2)$  and  $\Delta(he_0, he_1, he_3)$  share two hyperedges. In this case, we have  $Cov(\lambda_{\Delta_0}, \lambda_{\Delta_1}) = (p_i(ave))^4 - (p_i(ave))^6$ . Algorithm 3 depicts the specific steps for estimating the three types of triangles using the samples produced by *HyperSV*.

#### IV. HYPEREDGE-PAIR BASED SAMPLING AND ESTIMATING

Although *HyperSV* samples a hypergraph stream with no transformations, it has the following three problems in counting the three types of triangles explained as follows.

- 1) In estimating the counts of inner triangles, if many hyperedge pairs share at least three vertices in the sample set  $S$ , the estimation error is large as described in (7).
- 2) In estimating the counts of outer triangles and HE-Triangles, if many hyperedge pairs participate in the triangle formations in sample set  $S$ , the estimation errors of the outer triangles and He-Triangles are large as described in (9) and (11).
- 3) Since both the outer triangles and He-Triangles are formed by three completely connected hyperedge pairs while *HyperSV* just obtains the sample in the form of hyperedges, it is necessary to infer the probability of producing a hyperedge pair as  $p_i(ave)$ . Therefore, the estimation accuracy is largely influenced by  $p_i(ave)$  which may have a big deviation from the real value  $p_i(he\_pair)$  for  $he$ , resulting in inaccurate estimations of outer triangles and He-Triangles.

To further reduce estimation errors, we transform a hyperedge stream into a hyperedge-based pairwise graph stream. In the

**Algorithm 4:** *HyperSHE*.

---

**Input:** A pairwise graph stream  $G_h = \{e_1, \dots, e_n\}$   
**Output:** A set of edge samples  $S$

```

1  $S \leftarrow \{e_1, \dots, e_c\};$ 
2  $i \leftarrow c + 1;$ 
3 for  $e_i \in G_h$  do
4    $p_i^s = \frac{c}{i};$ 
5   Generate  $p_r$  randomly from  $(0, 1];$ 
6   if  $p_r \leq p_i^s$  then
7      $S \leftarrow S \cup e_i;$ 

```

---

pairwise graph stream, each hyperedge represents a vertex and two connected hyperedges form an edge. Based on the pairwise graph stream, this section proposes a hyperedge-based pairwise graph sampling algorithm to settle the above three problems, named *HyperSHE* and then estimates three types of triangles based on the obtained samples.

**A. Sampling Algorithm**

*HyperSHE's* process. Before the specific sampling process, it requires to transform the hyperedge stream into a hyperedge-based pairwise graph. Then, *HyperSHE* samples the pairwise graph as follows: The front  $c$  edges of the pairwise graph stream is stored in the static reservoir. From the  $i^{th}$  ( $i > c$ ) edge, the probability of the edge entering into the static reservoir is  $\frac{c}{i}$ . Once the edge is entering into the reservoir, one of the edges that have been stored in the static reservoir should be removed with probability  $p_o = \frac{1}{c}$ . Thus, when the  $i$ th edge is dealt with, the probability of each of the front  $i$  edges is stored in the static probability is  $p_i(e) = \frac{c}{i}$  according to (1). Algorithm 4 describes *HyperSHE's* process.

**B. Estimating Triangle Counts**

As described above, we compute the probability of forming inner triangles, outer triangles and He-Triangles to estimate their respective counts.

Inner triangles. Since the inner triangles are formed by the vertices in the same hyperedges, then the total count ( $T_\Delta(e_{he_0, he_1})$ ) of inner triangles stemmed from an edge is computed as  $T_\Delta(e_{he_0, he_1}) = N(\Delta)_{he_0} + N(\Delta)_{he_1} - N(\Delta)_{e_0}$  where  $N(\Delta)_{e_0}$  represents the count of the triangles formed by the common vertices shared by  $he_0$  and  $he_1$ . Furthermore, if a hyperedge, i.e.,  $he$  has  $deg(he)$  neighboring hyperedges, it is sampled with probability  $p(he) = deg(he) \times p_i(e)$  as  $deg(he)$  can be easily obtained from the hyperedge-based pairwise graph. Thus, the estimated count of inner triangles through *HyperSHE* is calculated as follows.

$$I_\Delta = \sum_{e(he_0, he_1) \in S} \left( \frac{N(\Delta)_{he_0}}{p(he_0)} + \frac{N(\Delta)_{he_1}}{p(he_1)} - \frac{N(\Delta)_{e_0}}{p_i(e)} \right) \quad (12)$$

Based on (12), let  $S(he)$  denote the hyperedge set formed by the hyperedges in  $S$ , the estimation variance of  $I_\Delta$  is described

as follows:

$$\begin{aligned} Var(I_\Delta) &= \sum_{he_0 \in S(he)} \left( \frac{N(\Delta)_{he_0}}{p(he_0)^2} [p(he_0) - p(he_0)^2] \right) \\ &+ \sum_{he_0 \in S(he)} \sum_{he_1 \in S(he)} Cov(he_0, he_1) - Var \left( \frac{N(\Delta)_{e_0}}{p_i(e)} \right) \end{aligned} \quad (13)$$

Two hyperedges in the hypergraphs streams are relevantly sampled in *HyperSHE* only if the two hyperedges are connected. Therefore, when  $he_0, he_1$  can not form an edge in the hyperedge-based pairwise graph,  $Cov(he_0, he_1) = 0$ ; Otherwise,  $Cov(he_0, he_1) = p_i(e) \times (1 - p_i(e))$ .

$$\begin{aligned} Var \left( \frac{N(\Delta)_{e_0}}{p_i(e)} \right) &= \frac{N(\Delta)_{e_0}}{p_i(e)^2} \sum_{e_0 \in S} \sum_{e_1 \in S} Cov(e_0, e_1) \\ &= \frac{N(\Delta)_{e_0}}{p(he)^2} \left[ \sum_{e_0 \in S} Var(e_0) + \sum_{e_0 \in S} \sum_{e_0 \in S, e_0 \neq e_1} Cov(e_0, e_1) \right] \\ &= \frac{N(\Delta)_{e_0}}{p_i(e)^2} \left[ (p_i(e) - p_i(e)^2) + \sum_{e_0 \in S} \sum_{e_1 \in S, e_0 \neq e_1} Cov(e_0, e_1) \right] \end{aligned} \quad (14)$$

Because whether one edge is sampled or not is irrelevant to another edge in *HyperSHE*, we have  $Cov(e_0, e_1) = 0$ .

Outer triangles. According to Theorem 2, outer triangles stem from three edges in the pairwise graph. Therefore, the probability of forming outer triangles by sampling is  $p(\Delta) = p(e)^3$  while the three edges can form  $n(\Delta)$  triangles. Thus, the estimated count of the outer triangles is estimated as follows.

$$O_\Delta = \sum_{e_0, e_1, e_2 \in S} \frac{n(\Delta)}{p_i(e)^3} \quad (15)$$

Accordingly, the estimation variance of  $O_\Delta$  is computed as follows, where  $S(\Delta_o^i)$  is the set of outer triangles formed by the sample set when *HyperSEV* is dealing with the  $i$ th hyperedge pair in  $G_h$ .

$$\begin{aligned} Var(O_\Delta) &= Var \left( \sum_{\Delta_0 \in S(\Delta_o^i)} \frac{1}{p_i(e)^3} \right) \\ &= \frac{1}{p_i(e)^6} \sum_{\Delta_0 \in S(\Delta)} \sum_{\Delta_1 \in S(\Delta)} Cov(\Delta_0, \Delta_1) \\ &= \frac{1}{p_i(e)^6} \left[ p_i(e)^3 - p_i(e)^6 \right. \\ &\quad \left. + \sum_{\Delta_0 \in S(\Delta_o^i)} \sum_{\Delta_1 \in S(\Delta_o^i), \Delta_0 \neq \Delta_1} Cov(\Delta_0, \Delta_1) \right] \end{aligned} \quad (16)$$

If the formations of two triangles, i.e.,  $\Delta_0$  and  $\Delta_1$  stem from six edges, we have  $Cov(\Delta_0, \Delta_1) = 0$ . Otherwise, the corresponding edges that generate the outer triangles at most share a common edge. In this case, we have  $Cov(\Delta_0, \Delta_1) = p_i(e)^5 - p_i(e)^6$ .

He-Triangles. As explained above, the He-Triangles are formed by three completely edges that may not generate outer triangles. Thus, it is necessary to recompute He-Triangles. Because the probability of forming a He-Triangle by *HyperSHE's* first stage sampling is  $p(\Delta) = p_i(e)^3$ , thus the total count of the triangles is estimated as follows.

$$E(HE_\Delta) = \sum_{e_0, e_1, e_2 \in S} \frac{1}{p_i(e)^3} \quad (17)$$

Therefore, the estimation variance of  $HE_\Delta^i(e)$  is expressed as follows, where  $S(\Delta_{he}^i)$  is the set of He-Triangles formed by the sample set when *HyperSEV* is dealing with the  $i$ th hyperedge pair in  $G_h$ .

$$\begin{aligned} Var(HE_\Delta) &= Var \left( \sum_{\Delta_0 \in S(\Delta_{he}^i)} \frac{1}{p_i(e)^3} \right) \\ &= \frac{1}{p_i(e)^6} \sum_{\Delta_0 \in S(\Delta_{he}^i)} \sum_{\Delta_1 \in S(\Delta_{he}^i)} Cov(\Delta_0, \Delta_1) \\ &= \frac{1}{p_i(e)^6} \left[ p_i(e)^3 - p_i(e)^6 \right. \\ &\quad \left. + \sum_{\Delta_0 \in S(\Delta_{he}^i)} \sum_{\Delta_1 \in S(\Delta_{he}^i), \Delta_0 \neq \Delta_1} Cov(\Delta_0, \Delta_1) \right] \quad (18) \end{aligned}$$

The proofs that (12), (15) and (17) can accurately estimate inner triangles, outer triangles and He-Triangles are similar to (6), (8) and (10) respectively. We omit the proofs due to page limits. Similar to  $Var(O_\Delta)$ , If the formations of two HE-Triangles, i.e.,  $\Delta_0$  and  $\Delta_1$  stem from six edges, we have  $Cov(\Delta_0, \Delta_1) = 0$ . Otherwise, the corresponding edges that generate the He-Triangle triangles at most share a common edge. In this case, we have  $Cov(\Delta_0, \Delta_1) = p_i(e)^5 - p_i(e)^6$ . Algorithm 5 presents the estimations of the three types of triangles using the samples produced by *HyperSHE*.

## V. DISCUSSION

*Comparison With Previous Work:* If each hyperedge in a hypergraph is formed by  $k$  vertices, the hypergraph can be called as a  $k$ -uniform hypergraph. Amit Chakrabarti et al. [31] proposed a suit of algorithms to find  $k$ -simplices in a  $k$ -uniform hypergraph where a  $k$ -simplex in  $H$  is formed by  $k + 1$  hyperedges among  $k + 1$  vertices in the hypergraph. Especially, when  $k = 2$ , 2-uniform hypergraph is a pairwise graph. Then, the methods in [31] can be used to estimate triangles in the pairwise graph rather than general hypergraphs. Although the methods in [31] focused on obtaining simplices in hypergraphs, they could not be used to estimate the triangles in general hypergraphs as explained follows.

- First, the methods in [31] are limited to estimate the count of the  $k$ -simplices in a  $k$ -uniform hypergraph. For a given hypergraph, it is necessary to divide the hypergraph into several parts as that: 2-uniform hypergraph, 3-uniform hypergraph, ..., and  $t$ -uniform hypergraph where  $t$  is the maximum number of vertices contained by a hyperedge in

### Algorithm 5: *HyperSHE\_Compute*( $\Delta$ ).

**Input:** Sample set  $S$

**Output:**  $I_\Delta^i(e)$ ,  $O_\Delta^i(e)$  and  $HE_\Delta^i(e)$

```

1 for  $e_h = (he_j, he_k) \in S$  do
2    $s_1 \leftarrow |V(he_j)|$ ;
3    $s_2 \leftarrow |V(he_k)|$ ;
4    $s_3 \leftarrow |V(he_j) \cap V(he_k)|$ ;
5   if  $s_1 \geq 3$  then
6      $N(\Delta)_{he_j} \leftarrow \frac{s_1 \times (s_1 - 1) \times (s_1 - 2)}{3 \times 2}$ ;
7   if  $s_2 \geq 3$  then
8      $N(\Delta)_{he_k} \leftarrow \frac{s_2 \times (s_2 - 1) \times (s_2 - 2)}{3 \times 2}$ ;
9   if  $s_3 \geq 3$  then
10     $N(\Delta)_{he_j, he_k} \leftarrow \frac{s_3 \times (s_3 - 1) \times (s_3 - 2)}{3 \times 2}$ ;
11     $I_\Delta = I_\Delta + (\frac{N(\Delta)_{he_j}}{p(he_j)} + \frac{N(\Delta)_{he_k}}{p(he_k)} - \frac{N(\Delta)_{he_j, he_k}}{p_i(e)});$ 
12 for  $e_i, e_j, e_k \in S$  do
13   if  $e_i, e_j, e_k$  form a hyperedge triangle then
14      $HE_\Delta \leftarrow HE_\Delta + \frac{1}{p_i(e)^3}$ ;
15      $O_\Delta \leftarrow O_\Delta + \frac{enum_\Delta(e_i, e_j, e_k)}{p_i(e)^3}$ ;

```

the hypergraph. Then, for  $2 \leq x \leq t$ , the  $x$ -uniform hypergraph is used to estimate the  $(x+1)$ -simplices which can be employed to compute the number of triangles. However, the methods using in this way consume lots of resources. As stated in [31], the space complexity of the methods is  $O(\epsilon^{-2} \log \delta^{-1} poly \log |V| \cdot \min\{\frac{|He|^{1+\frac{1}{k}}}{T}, \frac{|He|}{T^{\frac{2}{k+1}}}\})$ , labeled as  $O(k\text{-sim})$ , where  $T$  is number of  $k$ -simplices and  $\epsilon$  and  $\delta$  are the constants between 0 and 1. The proposed methods in this paper require a reservoir with static capacity  $C$  to store the sampled hyperedges and their space complexity is  $\sum_{1 \leq i \leq C} |S_i|$ , where  $C$  can be set small and then  $\sum_{1 \leq i \leq C} |S_i|$  is much smaller than  $O(k\text{-sim})$ . On the other hand, the proposed methods in this paper just process the hypergraph stream only once while the methods in [31] require to process the hypergraph stream for four times.

- Second, these methods can not discover the connections among the hyperedges with different number of vertices and then ignore the triangles formed by the vertices in such hyperedges. For example, a triangle  $\{u_1, u_2, u_4\}$  derived from the three hyperedges  $\{(u_1, u_2), (u_1, u_3, u_4), (u_2, u_4, u_6, u_7)\}$  can not be discovered by the methods in [31].

*Discussions of the Proposed Methods:* As this paper proposes two frameworks of counting the three types of triangles, this section discusses the preferences of two frameworks and further provides a guide for different application scenarios.

*HyperSV* and *HyperWSV* are able to process a hypergraph stream directly without any transformations and thus they can provide real-time estimations on the three types of triangles. Unlike the existing sampling algorithms which produce samples in the form of edges, *HyperSV* and *HyperWSV* obtain the hyperedge samples that can be employed to estimate the

TABLE II

SUMMARY OF HYPERGRAPH DATASETS WHERE INerTCnt, OuterTCnt AND HeTCnt DENOTE THE REAL NUMBER OF THE COUNTS OF INNER TRIANGLES, OUTER TRIANGLES AND HE-TRIANGLES RESPECTIVELY

| Hypergraph | V     | He   | InerTCnt        | OuterTCnt      | HeTCnt         |
|------------|-------|------|-----------------|----------------|----------------|
| NDC        | 5.5K  | 10K  | 9,174,264       | 9,927,341,116  | 916,298,660    |
| cooking    | 6.7K  | 39K  | 646,776         | 29,444,565     | 35,345,789     |
| walmart    | 88K   | 69K  | 993,329         | 42,854,114,838 | 36,154,730,221 |
| threads    | 125K  | 187K | 61,956          | 9,816,095,038  | 9,728,634,376  |
| DBLP       | 1.26M | 1.2M | 26,179,089      | 188,813,320    | 50,000,497     |
| stack      | 15M   | 1.1M | 389,312,478,665 | 12,786,275,522 | 194,285,609    |

three types of triangles by inferring the probabilities of forming different types of triangles respectively. On the other hand, although the probability of a hyperedge is certainly determined, the probability  $p_i(\text{he\_pair})$  of a hyperedge pair formed by two connected hyperedges is unknown prior to the specific sampling process. Although we infer the value of  $p_i(\text{he\_pair})$  by employing the average degree of a hyperedge, such way may increase the estimation errors in counting the outer triangles and He-Triangles.

*HyperSHE* provides more accuracy estimations on the counts of three types of triangles. *HyperSHE* produces samples of hyperedge pairs. Before the specific sampling process, it is required to transform the hypergraph stream into the hyperedge-based pairwise graphs, resulting in an increase of preprocessing time. On the other hand, we can get the number of neighboring hyperedges of a hyperedge easily. Thus, when *HyperSHE* is used to estimate the counts of the inner triangles, outer triangles and He-Triangles, the probabilities of forming three types of triangles can be inferred accurately to estimate the total counts of triangles accurately. Therefore, when an application requires more accurate estimations and can tolerate the non-real time processing, *HyperSHE* is a better choice.

## VI. EVALUATION

*Platform and Datasets:* We conduct our experiments on a computer with Intel(R) Xeon(R) Silver 4110 CPU @ 2.10GHz and 64GB memory. The experiments are conducted on the real-world datasets which are described as Table II. The six public datasets are frequently used in the existing researches [32], [33], [34] and also employed in the following experiments to evaluate our proposed frameworks.

*Baseline Algorithms:* We select the existing state-of-the-art sampling methods including GPS, Triest, and GSH to estimate inner and outer triangles to evaluate the efficiency of our proposed algorithms. As the existing sampling processes can not be used to estimate the three types of triangles simultaneously, we just compare our proposed two algorithms: *HyperSV* and *HyperSHE* to assess their performance in estimating He-Triangles. When a framework is used to estimate the triangle counts, it will be executed more than 10 times to obtain the mean value for evaluating its accuracy.

### A. Evaluating Hyperedge-Based Methods

This subsection describes the evaluation of the hyperedge-based sampling methods proposed in this paper from two aspects: estimation accuracy and processing time.

1) *Estimation Accuracy: Estimation error:* Suppose that  $X_{real}$  is the real value of an item and  $X_{estimated}$  is the value obtained by estimating, then the estimation error is computed as  $\frac{|X_{real} - X_{estimated}|}{X_{real}}$ . We evaluate the sampling methods when they are used to estimate the counts of the three types of triangles on the four real-world datasets to show the efficiency of our proposed methods.

Fig. 5 shows that the estimation error of the hyperedge-based sampling methods where *HyperSV*, *HyperWSV* and *HyperSHE* are the reservoir-based hyperedge sampling, reservoir-based weight sampling and hyperedge-pair based sampling. For the reservoir-based weight sampling, we set the values of parameter  $\alpha$  described in (2) as 0.2, 0.4, 0.6, 0.8 and 1 respectively. The sampling ratios are 0.03%, 0.05%, 0.07% and 0.05% of the total sizes over the four datasets. As shown in Fig. 5, *HyperSHE* exhibits the best and the estimation errors of *HyperSV* and *HyperWSV* are smaller than one in *NDC*, *walmart* and *threads* meaning that they are also effective in the three datasets with a relatively small sampling ratio. However, when to estimate the triangles of *stack*, *HyperSHE* is the only method that is able to estimate the three types of triangles accurately.

*Estimation variance:* Let  $k$  denote the number of executing a sampling algorithm when to estimate the counts of triangles and  $X_{mean}$  is the average value of the  $k$  estimated counts where  $k = 20$  in these experiments. Then, the estimation variance  $Var(X_{estimated})$  is estimated as that:  $Var(X_{estimated}) = \sum_{i=1, \dots, k} \frac{(X_{estimated}^i - X_{mean})^2}{k}$ , where  $X_{estimated}^i$  is the estimated value of the count at the  $i$ th execution. The estimation variance is used evaluate the stability of a sampling algorithm. The smaller estimation variance means the higher confidence of the sampling algorithm.

We evaluate the estimation variance by the confidence bounds as  $|X_{mean} \pm \sqrt{Var[X_{mean}]}|$  to compute the upper bound and lower bound [35]. As the estimation accuracies of inner triangles and outer triangles are similar, we choose the estimation variances of outer triangles and He-Triangles as the representatives of four datasets. The sampling ratios of *NDC*, *walmart*, *threads* and *DBLP* are 0.03%, 0.05%, 0.07% and 0.28% respectively. As shown in Figs. 6 and 7, most of the estimation values of the three types of sampling methods with 10 times for the four datasets locate at a reasonable field ranging from the lower bound to the upper bound shown by the dash lines. Overall, compared with *HyperSV* and *HyperWSV*, the estimation values of *HyperSHE* are more approximate to the real values identified by the dot lines.

2) *Processing Time:* Fig. 8 describes the processing time over the five datasets. *processFullHG* in the figure shows the processing time to obtain the real values of the three types of triangles over the five datasets while *generateHEPair* describes the times of transforming the five data hypergraphs into hyperedge pairs. The sampling ratios of the five datasets are 0.03%, 0.05%, 0.07%, 0.28% and 0.05% respectively. Fig. 8 shows that the sampling methods can cut down lots of time ranging from 1.7X to 1982X while they estimate the triangle counts at high accuracy as shown in Figs. 5, 6, and 7.

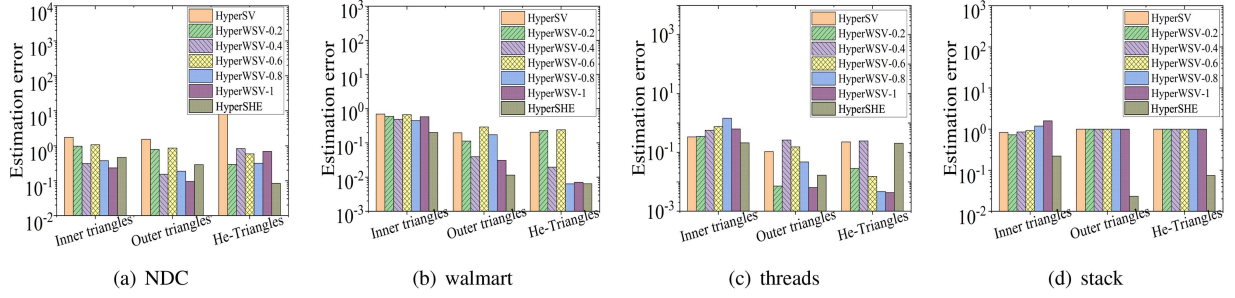


Fig. 5. Estimation error of hyperedge-based sampling methods over four datasets.

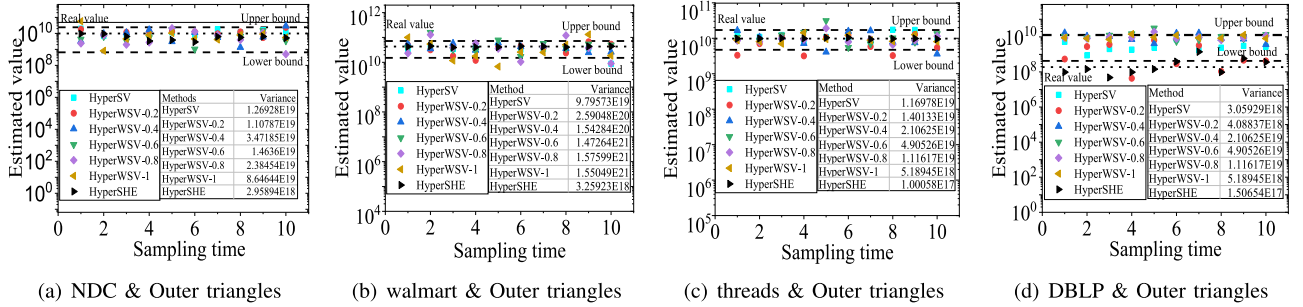


Fig. 6. Estimation values and variances for inner and outer triangles with 10 sampling times.

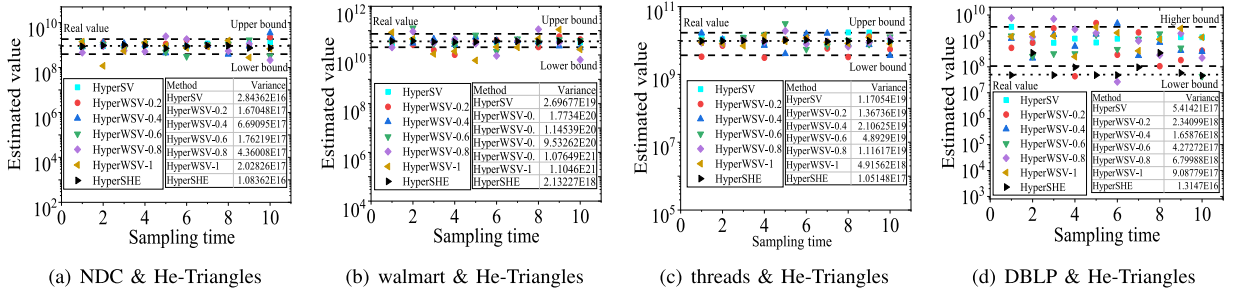


Fig. 7. Estimation values and variances for He-Triangles with 10 sampling times.

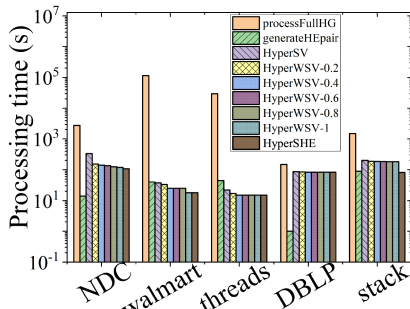


Fig. 8. Processing time over five datasets.

Furthermore, *HyperWSV* consumes less time than *HyperSV* as *HyperWSV* reduces the sampling probability of hyperedges with larger sizes. Although it is necessary to transform hypergraph streams into hyperedge pairs, the sum of processing time

consumed by *HyperSHE* and the transforming time is also less than *HyperSV* and *HyperWSV*. This is because two separate steps including obtaining hyperedge samples and discovering the connections among hyperedges are necessary for *HyperSV* and *HyperWSV* to estimate the total counts. *HyperSHE* belongs to one-passing sampling in which each hyperedge pair is processed for only one time and thus it consumes less time than *HyperSV* and *HyperWSV*.

### B. Comparing With the Existing Methods

*Estimation error of inner triangles:* Fig. 9(a) and (b) show that *HyperSV* and *HyperSHE* can estimate the counts of inner triangles much more accurately than *GPS*, *Triest* and *GSH* with the same capacity of the reservoir. Two reasons cause the larger estimation errors when estimating inner triangles by the three existing sampling methods. First, with the same capacity of

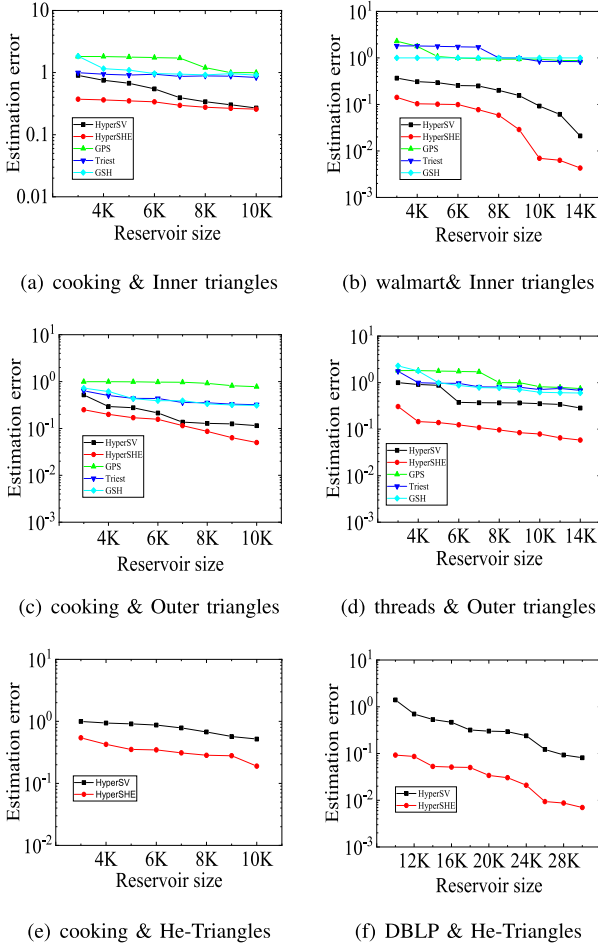


Fig. 9. Estimation errors of three types of triangles.

the reservoir, *HyperSV* and *HyperSHE* preserve the samples in the form of hyperedges and hyperedge pairs respectively while *GPS*, *Triest* and *GSH* preserve samples in the form of edges. As a hyperedge including  $n$  ( $n \geq 2$ ) vertices can represent  $\frac{n \times (n-1)}{2}$ , the number of samples obtained by *GPS*, *Triest* and *GSH* used to produce the inner triangles is much smaller than that obtained by *HyperSV* and *HyperSHE*. Second, the probability of forming an inner triangle obtained by the three existing sampling methods is considered smaller than the real value. If two vertices are included by more than one hyperedge, the edge formed by the two vertices should be sampled with a larger probability and therefore the probability of the inner triangles formed by this edge should be larger than that currently considered in the three existing sampling methods.

**Estimation error of outer triangles:** Fig. 9(c) and (d) describe the estimation errors caused by the five sampling methods. It is shown that *HyperSV* and *HyperSHE* perform better than the other three existing sampling methods. Since an outer triangle is formed by three edges which are produced by three corresponding hyperedges, the samples are required to cover different hyperedges. Because the existing sampling methods obtain the samples in the form of edges rather than hyperedges, these edges are either included by the same hyperedges or have

no common vertices to form a triangle. In other words, the samples obtained by them can not form enough outer triangles to estimate the total counts of the whole hypergraph stream. Furthermore, since *HyperSV* produces hyperedge samples, it has to infer the probabilities of producing hyperedge pairs before computing the probability of forming outer triangles, resulting in larger estimation errors than *HyperSHE* that produces the samples in the form of hyperedge pairs.

**Estimation error of He-Triangles:** As existing sampling methods focus on pairwise graph streams to produce edge samples, they can not estimate the counts of hyperedges that are formed by three completely connected hyperedges. Fig. 9(e) and (f) shows that *HyperSHE* estimates the He-Triangles more accurately than *HyperSV*. With the same capacity of reservoirs using by *HyperSHE* and *HyperSV*, it is more possible *HyperSHE* obtain more He-Triangles than *HyperSV* as explained above. Therefore, *HyperSHE* estimates the counts of He-Triangles with smaller estimation errors than *HyperSV*.

Fig. 9 shows that the existing sampling methods could not estimate the triangles in hypergraphs. The estimation errors with different sampling methods on pairwise graphs are close to one, inferring that with a small sampling budget, it is inadequate to use these methods to estimate triangles in hypergraphs. Conversely, our proposed methods can estimate the count of triangles in hypergraphs accurately as the estimated errors are much smaller than one with small sampling budgets.

## VII. RELATED WORK

To the best of our knowledge, we are the first to estimate the count of triangles defined in Section II in hypergraphs. The highlights of previous work on counting the triangles in pairwise graphs [5], [6], [36] and discovering the connection in hypergraphs are summarized below.

Given a graph stream formed by a large number of edges, estimating the count of triangles requires to learn the probability of producing a triangle which is formed by three connected edges [21], [24], [25]. Specifically, some existing works [21], [29], [37] designed sampling probabilities for the edges in a graph stream and then deduced the probabilities for forming triangles. As the triangles can be deduced by the wedges, some existing works [20], [38], [39] inferred the count of triangles by learning the probability of wedges to infer the probability of forming triangles to estimate the count of triangles. However, these sampling methods focus on pairwise graph streams to estimate the triangles counts while they do not consider the specific connections among vertices in hyperedges. Our proposed sampling and estimating frameworks discriminate the probabilities of forming the three types of triangles and further estimate their total counts respectively along with analyzing estimation variances.

On the other hand, many researches have been conducted on characterizing the connectivity in hypergraphs [40], [41]. Connectivity patterns among three hyperedges are learned to learn the motif of hypergraphs [1], [42]. Clustering coefficient are studied [2], [28], [43] to express the overall connectivity among hyperedges and can be used to predict the evolution

of a hypergraph. As analyzed in Section V, Chakrabarti et al. [31] proposed methods to count simplices rather than the triangles. Our study focuses on learning the three types of triangles formed by vertices, that is complementary to these studies for characterizing hypergraphs in the following aspects: (1) uncover the sources of vertices to form triangles which can reflect the impact of the triangles, (2) employ the connections among hyperedges to help discovering the connections among vertices and (3) estimate the total counts of the three types of triangles accurately which provide a basic tool to further analyze the global clustering coefficient.

## VIII. CONCLUSION

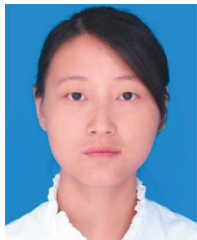
In this work, we are the first to introduce three types of triangles and then propose the sampling and estimating frameworks to accurately estimate the total counts of triangles. We summarize our work as follows.

- *Novel concepts:* We discover the triangles formed by different sources which bring in different impact on some applications. Then, we define them formally that is easy to classify a triangle in a hypergraph stream.
- *New sampling and estimating frameworks:* Since the existing sampling methods produce the samples in the form of edges by which the probability of forming an inner or outer triangle is inferred inaccurately, the newly sampling and estimating frameworks produce the samples in the form of hyperedges or the hyperedge pairs. We compute the probabilities of forming the three types of triangles respectively based on the samples to estimate the total counts and further analyze the estimation variances.
- *Experimental discoveries:* We conduct experiments on real-world datasets and discover that the three types of triangles do exist in real-world datasets. We first obtain the real values of the three types of triangles and then employ the real values to confirm the accuracy and efficiency of our estimated counts by using the proposed frameworks.

## REFERENCES

- [1] Y. Kook, J. Ko, and K. Shin, "Evolution of real-world hypergraphs: Patterns and models without oracles," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 272–281.
- [2] I. Amburg, N. Veldt, and A. Benson, "Clustering in graphs and hypergraphs with categorical edge labels," in *Proc. Web Conf.*, 2020, pp. 706–717.
- [3] E. Ramadan, A. Tarafdar, and A. Pothen, "A hypergraph model for the yeast protein complex network," in *Proc. 18th Int. Parallel Distrib. Process. Symp.*, 2004, Art. no. 189.
- [4] F. Klimm, C. M. Deane, and G. Reinert, "Hypergraphs for predicting essential genes using multiprotein complex data," *J. Complex Netw.*, vol. 9, no. 2, 2021, Art. no. cnaa028.
- [5] J. Shun and K. Tangwongsan, "Multicore triangle computations without tuning," in *Proc. IEEE 31st Int. Conf. Data Eng.*, 2015, pp. 149–160.
- [6] T. G. Kolda, A. Pinar, T. Plantenga, C. Seshadhri, and C. Task, "Counting triangles in massive graphs with MapReduce," *SIAM J. Sci. Comput.*, vol. 36, no. 5, pp. S48–S77, 2014.
- [7] L. Zhang, H. Jiang, F. Wang, D. Feng, and Y. Xie, "T-sample: A dual reservoir-based sampling method for characterizing large graph streams," in *Proc. IEEE 35th Int. Conf. Data Eng.*, 2019, pp. 1674–1677.
- [8] M. T. Do, S.-E. Yoon, B. Hooi, and K. Shin, "Structural patterns and generative models of real-world hypergraphs," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 176–186.
- [9] E. Estrada and J. A. Rodríguez-Velázquez, "Subgraph centrality and clustering in complex hyper-networks," *Physica A: Stat. Mechanics Appl.*, vol. 364, pp. 581–594, 2006.
- [10] X. Huang, H. Cheng, L. Qin, W. Tian, and J. X. Yu, "Querying K-truss community in large and dynamic graphs," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2014, pp. 1311–1322.
- [11] J. Wang and J. Cheng, "Truss decomposition in massive networks," *Proc. VLDB Endowment*, vol. 5, no. 9, pp. 812–823, 2012.
- [12] M. Gomez-Rodriguez, L. Song, N. Du, H. Zha, and B. Schölkopf, "Influence estimation and maximization in continuous-time diffusion networks," *ACM Trans. Inf. Syst.*, vol. 34, no. 2, pp. 1–33, 2016.
- [13] Z. Aghaei and S. Kianian, "Efficient influence spread estimation for influence maximization," *Social Netw. Anal. Mining*, vol. 10, no. 1, pp. 1–21, 2020.
- [14] P. Wang, G. Trimponias, H. Xu, and Y. Geng, "Luopan: Sampling-based load balancing in data center networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 1, pp. 133–145, Jan. 2019.
- [15] K. He, E. Rozner, K. Agarwal, W. Felter, J. Carter, and A. Akella, "Presto: Edge-based load balancing for fast datacenter networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 465–478, 2015.
- [16] A. McGregor, "Graph stream algorithms: A survey," *ACM SIGMOD Rec.*, vol. 43, no. 1, pp. 9–20, 2014.
- [17] A. Ching, S. Edunov, M. Kabiljo, D. Logothetis, and S. Muthukrishnan, "One trillion edges: Graph processing at facebook-scale," *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1804–1815, 2015.
- [18] K. Lee, H. Jo, J. Ko, S. Lim, and K. Shin, "SSumM: Sparse summarization of massive graphs," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 144–154.
- [19] R. Gemulla, W. Lehner, and P. J. Haas, "Maintaining bounded-size sample synopses of evolving datasets," *VLDB J. Int. J. Very Large Data Bases*, vol. 17, no. 2, pp. 173–201, 2008.
- [20] A. Pavan, K. Tangwongsan, S. Tirthapura, and K.-L. Wu, "Counting and sampling triangles from a graph stream," *Proc. VLDB Endowment*, vol. 6, no. 14, pp. 1870–1881, 2013.
- [21] N. K. Ahmed, N. Duffield, T. L. Willke, and R. A. Rossi, "On sampling from massive graph streams," *Proc. VLDB Endowment*, vol. 10, no. 11, pp. 1430–1441, 2017.
- [22] Z. Bar-Yossef, R. Kumar, and D. Sivakumar, "Reductions in streaming algorithms, with an application to counting triangles in graphs," in *Proc. 13th Annu. ACM-SIAM Symp. Discrete Algorithms*, 2002, pp. 623–632.
- [23] S. K. Bera and C. Seshadhri, "How the degeneracy helps for triangle counting in graph streams," in *Proc. 39th ACM SIGMOD-SIGACT-SIGAI Symp. Princ. Database Syst.*, 2020, pp. 457–467.
- [24] X. Lu and S. Bressan, "Sampling connected induced subgraphs uniformly at random," in *Proc. Int. Conf. Sci. Stat. Database Manage.*, 2012, pp. 195–212.
- [25] H. Jowhari and M. Ghodsi, "New streaming algorithms for counting triangles in graphs," in *Proc. Int. Comput. Combinatorics Conf.*, 2005, pp. 710–716.
- [26] C. Seshadhri, A. Pinar, and T. G. Kolda, "Wedge sampling for computing clustering coefficients and triangle counts on large graphs," *Stat. Anal. Data Mining: ASA Data Sci. J.*, vol. 7, no. 4, pp. 294–307, 2014.
- [27] N. K. Ahmed, J. Neville, and R. Kompella, "Network sampling: From static to streaming graphs," *ACM Trans. Knowl. Discov. Data (TKDD)*, vol. 8, no. 2, 2014, Art. no. 7.
- [28] S.-E. Yoon, H. Song, K. Shin, and Y. Yi, "How much and when do we need higher-order information in hypergraphs? a case study on hyperedge prediction," in *Proc. Web Conf.*, 2020, pp. 2627–2633.
- [29] N. K. Ahmed, N. Duffield, J. Neville, and R. Kompella, "Graph sample and hold: A framework for big-graph analytics," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2014, pp. 1446–1455.
- [30] L. D. Stefani, A. Epasto, M. Riondato, and E. Upfal, "TRIEST: Counting local and global triangles in fully dynamic streams with fixed memory size," *ACM Trans. Knowl. Discov. Data*, vol. 11, no. 4, 2017, Art. no. 43.
- [31] A. Chakrabarti and T. Haris, "Counting simplices in hypergraph streams," in *Proc. 30th Annu. Eur. Symp. Algorithms*, 2022, pp. 32:1–32:19.
- [32] I. Amburg, N. Veldt, and A. R. Benson, "Clustering in graphs and hypergraphs with categorical edge labels," in *Proc. Web Conf.*, 2020, pp. 706–717.
- [33] A. R. Benson, R. Abebe, M. T. Schaub, A. Jadbabaie, and J. Kleinberg, "Simplicial closure and higher-order link prediction," *Proc. Nat. Acad. Sci.*, vol. 115, pp. E11221–E11230, 2018.
- [34] M. Choe, J. Yoo, G. Lee, W. Baek, U. Kang, and K. Shin, "MiDaS: Representative sampling from real-world hypergraphs," in *Proc. ACM Web Conf.*, 2022, pp. 1080–1092.

- [35] Y. Tillé, *Sampling Algorithms*. Berlin, Germany: Springer, 2006.
- [36] K. Tangwongsan, A. Pavan, and S. Tirthapura, "Parallel triangle counting in massive streaming graphs," in *Proc. 22nd ACM Int. Conf. Inf. Knowl. Manage.*, 2013, pp. 781–786.
- [37] L. De Stefani, A. Epasto, M. Riondato, and E. Upfal, "TRIÈST: Counting local and global triangles in fully-dynamic streams with fixed memory size," *ACM Trans. Knowl. Discov. Data*, vol. 11, pp. 825–834, 2016.
- [38] Y. Lim and U. Kang, "MASCOT: Memory-efficient and accurate sampling for counting local triangles in graph streams," *ACM Trans. Knowl. Discov. Data*, vol. 1–28, pp. 685–694, 2015.
- [39] M. Jha, C. Seshadhri, and A. Pinar, "A space-efficient streaming algorithm for estimating transitivity and triangle counts using the birthday paradox," *ACM Trans. Knowl. Discov. Data*, vol. 9, no. 3, 2015, Art. no. 15.
- [40] D. Zhou, J. Huang, and B. Schölkopf, "Learning with hypergraphs: Clustering, classification, and embedding," in *Proc. Adv. Neural Inf. Process. Syst.*, 2006, pp. 1601–1608.
- [41] A. Bretto, "Hypergraphs: Basic concepts," in *Hypergraph Theory*. Berlin, Germany: Springer, 2013, pp. 1–21.
- [42] G. Lee, J. Ko, and K. Shin, "Hypergraph motifs: Concepts, algorithms, and discoveries," *Proc. VLDB Endowment*, vol. 13, no. 12, Jul. 2020, Art. no. 2256C2269.
- [43] A. R. Benson, R. Abebe, M. T. Schaub, A. Jadbabaie, and J. Kleinberg, "Simplicial closure and higher-order link prediction," *Proc. Nat. Acad. Sci.*, vol. 115, no. 48, pp. E11221–E11230, 2018.



**Lingling Zhang** received the PhD degree from the Huazhong University of Science and Technology (HUST). She is a postdoc research fellow in Computer Science of Beijing Institute of Technology (BIT). Her research interests include graph computations, online social networks, graph streams, computer networks, and distributed storage systems.



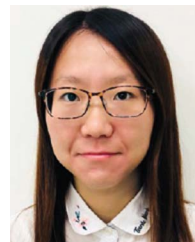
**Zhiwei Zhang** received the BS degree from the Renmin University of China, in 2010, and the PhD degree from the Chinese University of Hong Kong, in 2014. He is currently a professor with the Beijing Institute of Technology (BIT), Beijing, China. His research interests include federal learning, data pricing and transaction, distributed system, blockchain, and algorithm analysis.



**Guoren Wang** received the BS, MS, and PhD degrees from the Department of Computer Science, Northeastern University, China, in 1988, 1991, and 1996, respectively. Currently, he is a professor with the Beijing Institute of Technology (BIT), Beijing, China. His research interests include XML data management, query processing and optimization, bioinformatics, high dimensional indexing, parallel database systems, and cloud data management. He has published more than 100 research papers.



**Ye Yuan** received the BS, MS, and PhD degrees in computer science from Northeastern University, in 2004, 2007, and 2011, respectively. He is now a professor with the Beijing Institute of Technology (BIT), Beijing, China. His research interests include graph databases, probabilistic databases, and social network analysis.



**Kangfei Zhao** received the BE degree in the University of Science and Technology of China, in 2014 and the PhD degree in the Chinese University of Hong Kong, under the supervision of Prof. Jeffrey Xu Yu, in 2019. She is a senior researcher in Tencent AI Lab, Machine Learning Center. Her research interests include AI for database, graph analytics, graph data management.